

Relatório do Trabalho 3

Thamiris Yamate Fischer

Giuliano

Resumo—O seguinte relatório tem como objetivo apresentar a construção da Arquitetura Sagui, nas versões Vetorial e VLIW, assim como os códigos assembly desenvolvidos para cada um.

I. VETORIAL - SAGUI EM BANDO

A. Circuito

- 1) *Datapath:*
- 2) *ULA:*
- 3) *Sinais de Controle:*

B. Assembly

II. VLIW - SAGUI DE RABO LONGO

A. Processador

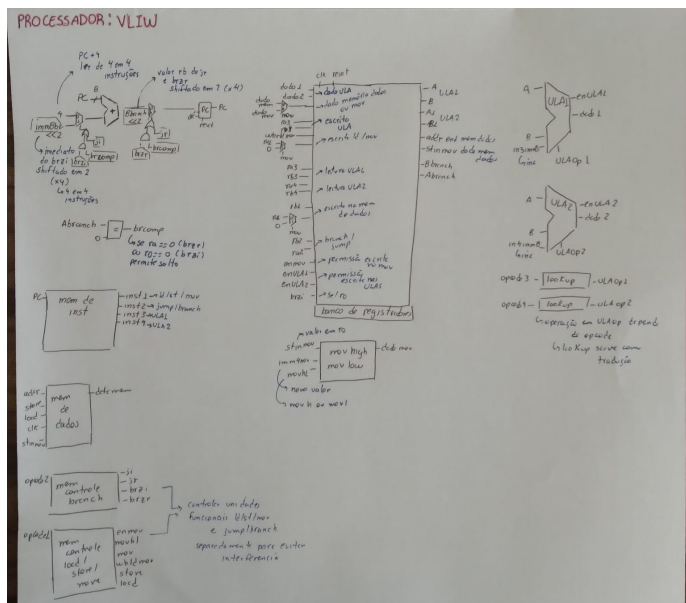


Figura 1. Datapath VLIW

1) Datapath:

- **Instruções:** As instruções são endereçadas por byte, ou seja, é necessário fazer $P_C = P_C + 4$ para saltar para a próxima instrução longa.
- **Banco de Registradores:** Como há quatro unidades funcionais que necessitam do banco de registradores, deve haver escritas e leituras destinadas para cada unidade funcional.

– Escrita:

- * **ULA:** São controlados por EnUla1 e EnUla2 para permitir ou não a escrita. Esses controles vem direto da ULA;

- * **ld / mov**: Escrita no banco de registradores para as operações de `load`, `movh` e `movl`. É controlado por `Enmov` e `wld` que vem da Memória de Controle `MemControleSLM`;

– **Leitura:**

- * **ULA**: Há uma leitura para cada ULA;
- * **st**: Pega o endereço em `rb1` para fazer o store;
- * **st/mov**: Pega o valor em `ra1` para armazenar na Memória de Dados ou em `r0` para manipular o valor com `movh` e `movl`;
- * **branch rb**: Pega o valor de salto em `rb2` para `brzr` ou `jr`;
- * **branch ra**: Pega o valor de comparação em `ra2` ou `r0` para decidir o salto em `brzr` ou `brzi`;

- **Movh / Movl:** Tem como input o valor de `r0` e o imediato. `movh` mantém os bits menos significativos de `r0` e substitui os bits mais significativos pelo imediato, `movl` faz o oposto. Esse circuito é controlado pelo `movhl`, se 1 é selecionado `movl` se 0, `movh`.
- **Branch:** Os valores tanto de imediato quanto do registrador `rb` nas operações de `branch` e `jump` são shiftados em 2, multiplicados por 4, para respeitar o tamanho das instruções longas. Além disso, há um componente de comparação, que para as operações `brzr` e `brzi` compara se `ra2` ou `r0`, respectivamente, são 0.
- **Extensão de sinal:** Há dois tipos de extensão, o do imediato de 2 bits para 8 bits, destinado à operação `inc`, e o do imediato de 4 bits para o de 4 bits para 8 bits, destinado às operações `movh`, `movl`, `ji` e `brzi`.

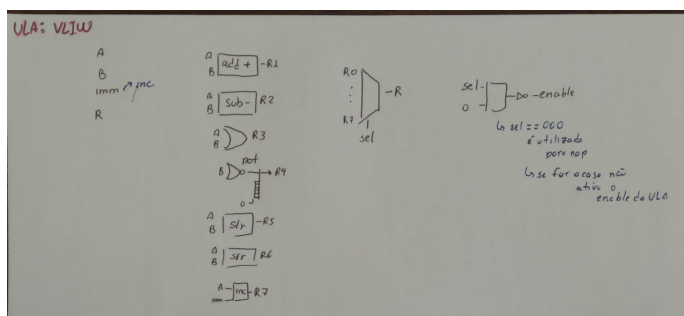


Figura 2. Diagrama de Caixas da ULA do VLIW

2) *ULA*:

- **Operações:** As operações 1 à 7 são destinadas às seguintes operações, respectivamente: add, sub, or, not, slr, srr e inc. A operação 0 é para nop, em que determina se o EnUla será ativado ou não, assim permitindo escrita no bando de registradores ou não;

- **slr e srr:** É feito manualmente com mux, assim se o valor de B exceder a capacidade do shifter ele deixa A zerado;
- **lookup ulaop:** Como cada ULA se comporta de maneira independente, as operações da ULA são determinados pelo Opcode, logo, é necessário uma tradução do Opcode para o Ulaop, isso é feito pelo lookup. Já que é utilizado uma memória para guardar essa tradução é possível alterar mais facilmente a ULA, se necessário.

3) **Sinais de Controle:** Há duas memórias de controle, uma para a unidade funcional de branch e jump e outra para a unidade funcional responsável por load, store e movh/movl.

Tabela I
MEMÓRIA DE CONTROLE: BRANCH

Opcode	Mnemonic	brzr	brzi	jr	ji
0000	brzr	1	0	0	0
0001	brzi	0	1	0	0
0010	jr	0	0	1	0
0011	ji	0	0	0	1
1111	nop	0	0	0	0

Tabela II
MEMÓRIA DE CONTROLE: LOAD, STORE, MOVH E MOVL

Opcode	Inst	load	store	wlm	mov	movhl	enmov
0000	ld	1	0	1	0	0	1
0001	st	0	1	0	0	0	0
0010	movh	0	0	1	1	0	1
0011	movl	0	0	1	1	1	1
1111	nop	0	0	0	0	0	0

B. Novas instruções

Duas instruções novas foram criadas:

- **ji:** Salta para o imediato indicado. Útil para jumps com distâncias pequenas, como para os loops que utilizam brzi. No código é utilizado para Inicializar os vetores R, A e B.
- **inc:** inc ra, imm, incrementa qualquer registrador ra com o imediato indicado, podendo ser 1 ou -1. É útil para inicializar qualquer registrador sem precisar do valor em r0 e, para o código, andar no vetor e diminuir os valores a colocar em A e B.

C. Assembly

```
;; Informacoes
;; E uma Arquitetura Harvard, ou seja, a memoria
de dados e separada da de instrucoes
;; Instrucao VLIW: ld/st/mv || branch/jump ||
ula || ula
```

```
;; Memaria: [0, 1]
;; #0 = contador
;; #1 = salto
;; Vetores: B[2-13] A[14-25] R[26-37]

;; Inicializar R
;; Colocar salto no #1
movh 1
nop
nop
nop

movl 9 ;; r0 = 0001 1001 = 25
nop
inc r1, 1 ;; r1 = 1
nop

st r0, r1 ;; #1 = 25 (salto do brzr)
nop
nop
nop

;; r1 = valor fim de R
movh 2 ;; r0 = 0010 0000
nop
sub r1, r1 ;; r1 = 0
nop

movl 5 ;; r0 = 0010 0101 = 37
nop
nop
nop

nop
nop
add r1, r0 ;; r1 = 37
nop

;; r0 = 12 -> contador
movh 0
nop
nop
nop

movl 12 ;; r0 = 0000 1100 = 12
nop
nop
nop

;; Armazenar contador em #0
st r2, r0 ;; #0 = 12 (contador)
nop
nop
nop
nop

;; loop
nop
brzi 3
nop
nop

st r2, r1 ;; R[r1] = 0
nop
nop
inc r0, -1 ;; Diminui contador

nop
ji -2
inc r1, -1 ;; Diminui ponteiro
nop

;; fim loop
;; r0 = r2 = r3 = 0 | r1 = 25 (fim de A )
```

```

;; Inicializar A e B
;; r2 = ultimo valor de B + 1
movh 2 ;; r0 = 0010 0000 = 32
nop
nop
nop

nop
nop
nop
add r2, r0 ;; r2 = 32
nop

;; r3 = ultima pos de B + 1
movh 0 ;; r0 = 0000 0000
nop
nop
nop

movl 15 ;; r0 = 0000 1111 = 15
nop
nop
nop

nop
nop
nop
add r3, r0 ;; r3 = 15
nop

;; r0 = ultimo valor de A
movl 11 ;; r0 = 0000 1011 = 11
nop
nop
nop

;; r0 = 11 | r1 = 25 | r2 = 32 | r3 = 15

;; loop
nop
brzi 4
nop
nop

st r0, r1 ;; A[r1] = r0
nop
inc r2, -1
inc r3, -1

st r2, r3 ;; B[r3] = r2
nop
inc r0, -1
inc r1, -1

nop
ji -3
nop
nop

;; fim loop

;; Colocar primeira pos de B >> #3 = 20 e
primeira pos de A #14 = 0
st r0, r1
nop
inc r2, -1
inc r3, -1

;;20
st r2, r3
nop
nop
nop

;; r0 = 0 | r1 = 14 | r2 = 20 | r3 = 3
;; r1 e r3 sao ponteiros
;; Soma

```

```

nop
nop
inc r1, -1 ;; r1 = 13 -> ajuste
nop

;; loop
ld r2, r3 ;; r2 = B[r3]
nop
sub r0, r0
inc r1, 1 ;; andar pos em A

ld r0, r1 ;; r0 = A[r1]
nop
nop
inc r3, 1 ;; andar pos em B

nop
nop
nop
add r2, r0 ;; r2 = val.A + val.B
nop

;; Os bits mais significativos de r0 nunca
serao diferentes de 0000, ja que os
valores do vetor A vao de 0 -> 11

movl -4 ;; r0 = 0000 1100 = 12
nop
nop
nop

nop
nop
nop
add r1, r0 ;; Ir para R
nop

st r2, r1 ;; Guardar soma em R
nop
nop
nop

nop
nop
nop
sub r1, r0 ;; Retornar para A
nop

;; Pegar contador e valor brzr

movl 0 ;; r0 = 0000 0000
nop
nop
nop

ld r2, r0 ;; r2 = contador
nop
nop
nop

nop
nop
nop
inc r2, -1 ;; contador--
nop

st r2, r0 ;; armazena contador
nop
nop
nop
nop

movl 1 ;; r0 = 0000 0001
nop
nop
nop
nop

ld r2, r0 ;; r2 = val.salto
nop

```

```

    nop
    nop

    movl 0 ;; r0 = 0000 0000
    nop
    nop
    nop

    ld r0, r0 ;; r0 = contador
    nop
    nop
    nop

    nop
    nop
    not r0, r0 ;;
    nop

    nop
    brzr r0, r2
    nop
    nop

;; fim loop
nop
ji 0 ;; halt
nop
nop

```

Listing 1. Assembly do VLIW